SystemVerilog and UVM Practice Journey

Phase 02 — Arrays and Collections

Bibhu Asish Panda

November 10, 2026

Contents

2 Phase 02 — Arrays and Collections 1

2.1 Overview 1

2.2 Topics Covered 1

2.2.1 Array Types and Operations 1

2.3 Key Learning Areas 2

2.4 Practical Work 2

2.5 Learning Outcomes 2

2.6 Tools 2

2.7 SV-013 — Fixed Arrays 3

2.7.1 Fixed Arrays 3

2.7.2 1D Fixed Array 3

2.7.3 Multidimensional Fixed Array 3

2.7.4 `foreach` 4

2.7.5 `$size` 4

2.7.6 Why Fixed Arrays Matter in Verification 5

2.7.7 Complete Runnable Code 5

2.7.8 Expected Output 5

2.7.9 Key Syntax 6

2.8 SV-014 — Dynamic Arrays 7

2.8.1 Dynamic Arrays 7

2.8.2 `new[]` 7

2.8.3 Initialization 7

2.8.4 Resize 7

2.8.5 `size()` 8

2.8.6 `delete()` 8

2.8.7 Dynamic Array Operations 8

2.8.8 Why Dynamic Arrays Matter in Verification 9

2.8.9 Complete Runnable Code 9

2.8.10 Expected Output 10

2.8.11 Key Syntax 10

2.9 SV-015 — Dynamic Array Copying 11

2.9.1 Dynamic Array Copying 11

2.9.2 `new[size](array)` 11

2.9.3 Preserving Elements 11

2.9.4 Increasing the Array Size 12

2.9.5 Decreasing the Array Size 12

2.9.6 Why This Matters in Verification 12

2.9.7 Complete Runnable Code . . . 13
2.9.8 Expected Output . . . 13
2.9.9 Key Syntax . . . 14
2.10 SV-016 — Queue . . . 15
2.10.1 Introduction . . . 15
2.10.2 Verification Relevance . . . 15
2.10.3 Complete Runnable Code . . . 16
2.10.4 Expected Output . . . 16
2.10.5 Key Takeaways . . . 17
2.11 SV-017 — Queue Methods . . . 18
2.11.1 Introduction . . . 18
2.11.2 Important Queue Methods . . . 18
2.11.3 Indexing . . . 18
2.11.4 Delete . . . 18
2.11.5 Size . . . 18
2.11.6 Verification Relevance . . . 18
2.11.7 Complete Runnable Code . . . 19
2.11.8 Expected Output . . . 19
2.11.9 Key Takeaways . . . 20
2.12 SV-018 — Associative Arrays . . . 21
2.12.1 Introduction . . . 21
2.12.2 Integer Indexing . . . 21
2.12.3 Sparse Storage . . . 21
2.12.4 Verification Relevance . . . 21
2.12.5 Complete Runnable Code . . . 21
2.12.6 Expected Output . . . 22
2.12.7 Key Takeaways . . . 22
2.13 SV-019 — Associative Array Methods . . . 23
2.13.1 Introduction . . . 23
2.13.2 Important Methods . . . 23
2.13.3 `first()` and `last()` . . . 23
2.13.4 `next()` and `prev()` . . . 23
2.13.5 `exists()` . . . 24
2.13.6 `num()` . . . 24
2.13.7 `delete()` . . . 24
2.13.8 Verification Relevance . . . 24
2.13.9 Complete Runnable Code . . . 24
2.13.10 Expected Output . . . 25
2.13.11 Key Takeaways . . . 26
2.14 SV-020 — Array Locator Methods . . . 27
2.14.1 Introduction . . . 27
2.14.2 `find()` . . . 27
2.14.3 `find_index()` . . . 27
2.14.4 `find_first()` . . . 27
2.14.5 `find_last()` . . . 27
2.14.6 The `with` Clause . . . 28

2.14.7 Verification Relevance . . . 28
2.14.8 Complete Runnable Code . . . 28
2.14.9 Expected Output . . . 29
2.14.10 Key Takeaways . . . 29
2.15 SV-021 — Array Ordering . . . 30
2.15.1 Introduction . . . 30
2.15.2 `sort()` . . . 30
2.15.3 `rsort()` . . . 30
2.15.4 `reverse()` . . . 30
2.15.5 `shuffle()` . . . 30
2.15.6 Verification Relevance . . . 31
2.15.7 Complete Runnable Code . . . 31
2.15.8 Expected Output . . . 32
2.15.9 Key Takeaways . . . 32
2.16 SV-022 — Array Reduction . . . 33
2.16.1 Introduction . . . 33
2.16.2 `sum()` . . . 33
2.16.3 `product()` . . . 33
2.16.4 `and()` . . . 33
2.16.5 `or()` . . . 34
2.16.6 `xor()` . . . 34
2.16.7 Verification Relevance . . . 34
2.16.8 Complete Runnable Code . . . 34
2.16.9 Expected Output . . . 35
2.16.10 Key Takeaways . . . 35
2.17 SV-023 — Array Iteration . . . 36
2.17.1 Introduction . . . 36
2.17.2 `foreach` . . . 36
2.17.3 Iterator Variables . . . 36
2.17.4 Array Traversal . . . 36
2.17.5 Verification Relevance . . . 36
2.17.6 Complete Runnable Code . . . 37
2.17.7 Expected Output . . . 37
2.17.8 Key Takeaways . . . 37
2.18 SV-024—Assignment-Patterns . . . 38
2.18.1 Introduction . . . 38
2.18.2 Nested Assignment Patterns . . . 38
2.18.3 Struct Assignment Pattern . . . 38
2.18.4 Verification Relevance . . . 38
2.18.5 Complete Runnable Code . . . 38
2.18.6 Expected Output . . . 39

Chapter 2

Phase 02 — Arrays and Collections

2.1 Overview

This phase explores SystemVerilog array types and collection operations used to store, access, modify, and process multiple values. It covers fixed arrays, dynamic arrays, queues, associative arrays, and built-in array methods commonly used in testbench development and verification.

2.2 Topics Covered

2.2.1 Array Types and Operations

No.	Topic	Concepts Covered
SV-013	Fixed Arrays	1D arrays, multidimensional arrays, initialization
SV-014	Dynamic Arrays	`new[]`, resizing, `delete`, `size`
SV-015	Dynamic Array Copying	`new[size](array)`, preserving elements
SV-016	Queue	`[$]`, `push_front`, `push_back`, `pop_front`, `pop_back`
SV-017	Queue Methods	`insert`, `delete`, `size`, indexing
SV-018	Associative Arrays	Integer indexing, string indexing, sparse storage
SV-019	Associative Array Methods	`first`, `last`, `next`, `prev`, `exists`, `num`, `delete`
SV-020	Array Locator Methods	`find`, `find_index`, `find_first`, `find_last`
SV-021	Array Ordering	`sort`, `rsort`, `reverse`, `shuffle`
SV-022	Array Reduction	`sum`, `product`, `and`, `or`, `xor`
SV-023	Array Iteration	`foreach`, `iterator variables`, `array traversal`
SV-024	Assignment Patterns	`'{ }`, `nested assignment patterns`

2.3 Key Learning Areas

- **Array management:** Declare, initialize, access, and modify fixed and dynamic arrays.
- **Dynamic storage:** Resize arrays, preserve existing elements, and manage array memory.
- **Collection handling:** Use queues for ordered data and associative arrays for key-based, sparse storage.
- **Array processing:** Locate elements, reorder collections, and perform reduction operations.
- **Iteration:** Traverse array elements using `foreach` and iterator variables.
- **Data initialization:** Use assignment patterns to initialize arrays and nested collections efficiently.

2.4 Practical Work

Each topic is practiced through runnable SystemVerilog examples. Vivado simulation is used to observe array operations, inspect outputs, and verify expected behavior.

2.5 Learning Outcomes

After completing this phase, you should be able to: Choose suitable array and collection types for different requirements. Perform array resizing, copying, insertion, deletion, and indexing. Apply built-in locator, ordering, and reduction methods. Traverse and process collections efficiently. Initialize arrays using simple and nested assignment patterns.

2.6 Tools

- **Language:** SystemVerilog
- **Simulator:** AMD Vivado
- **Practice:** Runnable code examples, simulation outputs, and GitHub learning notes

2.7 SV-013 — Fixed Arrays

2.7.1 Fixed Arrays

A fixed array has a size that is determined at compile time and remains constant during simulation.

SystemVerilog supports one-dimensional and multidimensional fixed arrays.

Fixed arrays are commonly used in verification for:

- Test data
- Expected results
- Lookup tables
- Memory models
- Groups of related values
- Transaction data

2.7.2 1D Fixed Array

A one-dimensional fixed array uses one index.

```
int numbers [5];
```

This creates five elements:

```
numbers[0]
numbers[1]
numbers[2]
numbers[3]
numbers[4]
```

The size is fixed at `5`.

Initialization: A fixed array can be initialized using an array assignment pattern:

```
numbers = '{10, 20, 30, 40, 50};
```

The values are assigned to the array elements in order.

2.7.3 Multidimensional Fixed Array

A multidimensional array uses multiple indices.

```
int matrix [2][3];
```

This creates:

- 2 rows
- 3 columns
- 6 total elements

The elements are accessed using two indices:

```
matrix[0][0]
matrix[0][1]
matrix[0][2]

matrix[1][0]
matrix[1][1]
matrix[1][2]
```

It can be initialized using nested assignment patterns:

```
matrix = '{
    '{1, 2, 3},
    '{4, 5, 6}
};
```

2.7.4 `foreach`

The `foreach` loop provides a convenient way to iterate through array elements. For a 1D array:

```
foreach (numbers[i])
    $display("%0d", numbers[i]);
```

For a 2D array:

```
foreach (matrix[i, j])
    $display("%0d", matrix[i][j]);
```

The loop variables automatically represent the valid indices of the array.

2.7.5 `$size`

`$size` can be used to determine the number of elements in an array.

For a 1D array:

```
$size(numbers)
```

returns:

```
5
```

For a multidimensional array, the dimension can be specified:

```
$size(matrix, 1)
```

returns the size of the first dimension:

```
2
```

and:

```
$size(matrix, 2)
```

returns the size of the second dimension:

```
3
```

2.7.6 Why Fixed Arrays Matter in Verification

Fixed arrays are useful when the required size is known beforehand.

Examples include:

```
Expected data:      10, 20, 30, 40, 50
Memory model:       2D/3D storage
Packet data:        Fixed number of bytes
Test vectors:       Predefined input values
Lookup tables:      Constant-size reference data
```

They provide predictable storage and straightforward indexing.

2.7.7 Complete Runnable Code

Code type: SystemVerilog simulation/demo — no DUT.

```
module sv_013_fixed_arrays;

    int numbers [5];          // 1D fixed array
    int matrix [2][3];        // 2D fixed array

    initial begin
        // 1D array initialization
        numbers = '{10, 20, 30, 40, 50};
        $display("1D Fixed Array:");
        foreach (numbers[i])
            $display("numbers[%0d] = %0d", i, numbers[i]);

        // 2D array initialization
        matrix = '{
            '{1, 2, 3},
            '{4, 5, 6}
        };
        $display("\n2D Fixed Array:");
        foreach (matrix[i, j])
            $display("matrix[%0d][%0d] = %0d", i, j, matrix[i][j]);

        // Array dimensions
        $display("\nArray Information:");
        $display("numbers size = %0d", $size(numbers));
        $display("matrix rows  = %0d", $size(matrix, 1));
        $display("matrix cols  = %0d", $size(matrix, 2));
    end
endmodule
```

2.7.8 Expected Output

```
1D Fixed Array:
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
```

```
numbers[3] = 40
numbers[4] = 50

2D Fixed Array:
matrix[0][0] = 1
matrix[0][1] = 2
matrix[0][2] = 3
matrix[1][0] = 4
matrix[1][1] = 5
matrix[1][2] = 6

Array Information:
numbers size = 5
matrix rows  = 2
matrix cols  = 3
```

2.7.9 Key Syntax

1D Fixed Array:

```
data_type array_name [size];
```

Example:

```
int numbers [5];
```

2D Fixed Array:

```
data_type array_name [rows][columns];
```

Example:

```
int matrix [2][3];
```

Array Initialization:

```
numbers = '{10, 20, 30, 40, 50};
```

Multidimensional Initialization:

```
matrix = '{
    '{1, 2, 3},
    '{4, 5, 6}
};
```

Iteration:

```
foreach (array[i])
    ...
```

For multidimensional arrays:

```
foreach (array[i, j])
    ...
```

Important Point: A **fixed array has a compile-time-defined size**. Its number of elements does not change during simulation. This is the foundation for the next SystemVerilog array types, where we will move from fixed-size storage to more flexible collections.

2.8 SV-014 — Dynamic Arrays

2.8.1 Dynamic Arrays

A **dynamic array** is an array whose size can be determined and changed during simulation.

Unlike a fixed array:

```
int data [5];
```

a dynamic array is declared without a size:

```
int data[];
```

Memory is allocated later using `new[]`.

Dynamic arrays are useful in verification when the required number of elements is not known at compile time.

2.8.2 `new[]`

The `new[]` operator allocates memory for a dynamic array.

```
data = new[5];
```

This creates five elements:

```
data[0]
data[1]
data[2]
data[3]
data[4]
```

The current number of elements can be checked using:

```
data.size()
```

2.8.3 Initialization

After allocating the array, its elements can be assigned normally:

```
foreach (data[i])
    data[i] = (i + 1) * 10;
```

This produces:

```
10, 20, 30, 40, 50
```

2.8.4 Resize

A dynamic array can be resized using another `new[]` operation:

```
data = new[8](data);
```

The first argument specifies the new size:

```
new[8]
```

The second argument is the existing array:

```
(data)
```

When resizing this way, the existing elements are copied into the new array up to the available size.

For example:

```
Before:
10 20 30 40 50

After resizing to 8:
10 20 30 40 50 0 0 0
```

The newly created elements receive their default value.

2.8.5 `size()`

The `size()` method returns the current number of elements.

```
data.size()
```

For example:

```
After new[5]  -> 5
After new[8]  -> 8
```

After deleting the array:

```
data.size() -> 0
```

2.8.6 `delete()`

The `delete()` method removes all elements from a dynamic array:

```
data.delete();
```

After deletion, the dynamic array has zero elements.

This is useful when dynamically allocated testbench data is no longer required.

2.8.7 Dynamic Array Operations

Operation	Purpose
`int data[];`	Declare dynamic array
`new[N]`	Allocate `N` elements
`new[N](data)`	Resize and preserve existing values
`data.size()`	Return current number of elements
`data.delete()`	Delete all elements

2.8.8 Why Dynamic Arrays Matter in Verification

Dynamic arrays are useful when testbench data is variable in size, for example:

- Variable-length packets
- Test vectors
- Transaction data
- Memory models
- Runtime-generated stimulus
- Collections whose size is determined during simulation

They provide more flexibility than fixed arrays while still using indexed array access.

2.8.9 Complete Runnable Code

Code type: SystemVerilog simulation/demo — no DUT.

```
module sv_014_dynamic_arrays;

    int data[];

    initial begin
        // Allocate 5 elements
        data = new[5];

        // Initialize values
        foreach (data[i])
            data[i] = (i + 1) * 10;

        $display("Initial size = %0d", data.size());

        foreach (data[i])
            $display("data[%0d] = %0d", i, data[i]);

        // Resize to 8 elements and preserve old values
        data = new[8](data);

        $display("\nAfter resize = %0d", data.size());

        foreach (data[i])
            $display("data[%0d] = %0d", i, data[i]);

        // Delete the dynamic array
        data.delete();

        $display("\nAfter delete = %0d", data.size());
    end

endmodule
```

2.8.10 Expected Output

```
Initial size = 5
data[0] = 10
data[1] = 20
data[2] = 30
data[3] = 40
data[4] = 50

After resize = 8
data[0] = 10
data[1] = 20
data[2] = 30
data[3] = 40
data[4] = 50
data[5] = 0
data[6] = 0
data[7] = 0

After delete = 0
```

2.8.11 Key Syntax

Declaration:

```
data_type array_name[];
```

Allocation:

```
array_name = new[size];
```

Resize:

```
array_name = new[new_size](array_name);
```

Size:

```
array_name.size()
```

Delete:

```
array_name.delete();
```

The key difference from a fixed array is:

> **A dynamic array has a runtime-defined size that can be changed using `new[]`.**

2.9 SV-015 — Dynamic Array Copying

2.9.1 Dynamic Array Copying

SystemVerilog allows a dynamic array to be allocated with a specific size while copying elements from an existing dynamic array.

The syntax is:

```
new[size](array)
```

For example:

```
copy = new[8](source);
```

This creates a new dynamic array with **8 elements** and copies the existing elements from `source` into it.

2.9.2 `new[size](array)`

Consider a source array containing:

```
10, 20, 30, 40, 50
```

If we execute:

```
copy = new[8](source);
```

the new array contains:

```
10, 20, 30, 40, 50, 0, 0, 0
```

The existing five elements are preserved, while the additional elements receive their default value.

Therefore:

```
Source size = 5
New size    = 8
```

2.9.3 Preserving Elements

The important feature of:

```
new[new_size](existing_array)
```

is that the existing elements are copied into the newly allocated array.

For example:

```
source = new[5];

foreach (source[i])
    source[i] = (i + 1) * 10;

copy = new[8](source);
```

The resulting array is:

```
copy[0] = 10
copy[1] = 20
copy[2] = 30
copy[3] = 40
copy[4] = 50
copy[5] = 0
copy[6] = 0
copy[7] = 0
```

The original `source` array is not resized or modified by this operation.

2.9.4 Increasing the Array Size

If the new size is larger than the source:

```
copy = new[8](source);
```

existing elements are preserved and additional elements are initialized to their default value.

2.9.5 Decreasing the Array Size

The same mechanism can reduce the number of elements:

```
copy = new[3](source);
```

If `source` contains:

```
10, 20, 30, 40, 50
```

then `copy` contains:

```
10, 20, 30
```

Only the elements that fit into the new array are preserved.

2.9.6 Why This Matters in Verification

Dynamic array copying is useful when:

- Test data needs to grow during simulation
- Existing transaction data must be preserved
- Variable-length packets are being constructed
- Test vectors need resizing
- A larger or smaller working array is required

The key concept is:

`new[size](array)` allocates a new dynamic array and copies the existing elements into it, up to the available size.

2.9.7 Complete Runnable Code

Code type: SystemVerilog simulation/demo — no DUT.

```
module sv_015_dynamic_array_copying;
    int source[];
    int copy[];

    initial begin
        // Create and initialize source array
        source = new[5];

        foreach (source[i])
            source[i] = (i + 1) * 10;

        $display("Source array:");
        foreach (source[i])
            $display("source[%0d] = %0d", i, source[i]);

        // Copy source into a larger dynamic array
        copy = new[8](source);

        $display("\nCopied array:");
        foreach (copy[i])
            $display("copy[%0d] = %0d", i, copy[i]);

        $display("\nSource size = %0d", source.size());
        $display("Copy size   = %0d", copy.size());
    end
endmodule
```

2.9.8 Expected Output

```
Source array:
source[0] = 10
source[1] = 20
source[2] = 30
source[3] = 40
source[4] = 50

Copied array:
copy[0] = 10
copy[1] = 20
copy[2] = 30
copy[3] = 40
copy[4] = 50
copy[5] = 0
copy[6] = 0
copy[7] = 0

Source size = 5
Copy size   = 8
```

2.9.9 Key Syntax

Allocate and copy:

```
copy = new[8](source);
```

Increase size while preserving elements:

```
array = new[larger_size](array);
```

Decrease size while preserving elements that fit:

```
array = new[smaller_size](array);
```

Remember:

```
new[size](array)
     |      |
     |      +-- existing array to copy
     +--------- new array size
```

This operation creates a **new dynamic array** and preserves the source elements that fit within the requested size.

2.10 SV-016 — Queue

2.10.1 Introduction

A **queue** in SystemVerilog is an ordered collection whose size can change dynamically during simulation. Queues are declared using `[$]` and are commonly used in verification environments to store transactions, stimulus, expected data, or pending operations.

Queue Declaration:

```
int queue[$];
```

Here:

- `int` is the element type.
- `queue` is the queue variable.
- `[$]` indicates an unbounded queue.

Important Queue Operations:

Operation	Purpose
`push_front(value)`	Adds an element at the beginning
`push_back(value)`	Adds an element at the end
`pop_front()`	Removes and returns the first element
`pop_back()`	Removes and returns the last element
`size()`	Returns the current number of elements

Operation Example:

Starting with an empty queue:

```
push_back(20)  -> {20}
push_back(30)  -> {20, 30}
push_front(10) -> {10, 20, 30}

pop_front()    -> returns 10 -> {20, 30}
pop_back()     -> returns 30 -> {20}
```

`pop_front()` and `pop_back()` both **remove an element and return its value**, which makes queues useful for FIFO/LIFO-style verification operations.

2.10.2 Verification Relevance

Queues are frequently used in verification to:

- Store generated transactions.
- Maintain expected results.
- Implement FIFO-like behavior.
- Pass stimulus between verification components.
- Compare received transactions against previously stored expected transactions.

2.10.3 Complete Runnable Code

```
module sv_016_queue;

    int queue[$];
    int value;

    initial begin
        // Add elements
        queue.push_back(20);
        queue.push_back(30);
        queue.push_front(10);

        $display("After push operations:");
        $display("queue = %p", queue);

        // Remove first element
        value = queue.pop_front();
        $display("\npop_front = %0d", value);
        $display("queue     = %p", queue);

        // Remove last element
        value = queue.pop_back();
        $display("\npop_back  = %0d", value);
        $display("queue     = %p", queue);

        // Add elements again
        queue.push_front(5);
        queue.push_back(40);

        $display("\nAfter adding again:");
        $display("queue = %p", queue);
        $display("size  = %0d", queue.size());
    end

endmodule
```

2.10.4 Expected Output

```
After push operations:
queue = '{10, 20, 30}

pop_front = 10
queue     = '{20, 30}

pop_back  = 30
queue     = '{20}

After adding again:
queue = '{5, 20, 40}
size  = 3
```

2.10.5 Key Takeaways

- `[$]` declares an unbounded queue.
- `push_front()` inserts at the beginning.
- `push_back()` inserts at the end.
- `pop_front()` removes and returns the first element.
- `pop_back()` removes and returns the last element.
- `size()` reports the current queue size.
- Queues are particularly useful for storing and managing verification transactions.

2.11 SV-017 — Queue Methods

2.11.1 Introduction

SystemVerilog queues provide methods for inserting, deleting, and inspecting elements. Queue indexing can also be used to access a specific element directly. These operations are useful in verification for dynamically managing collections of transactions or expected data.

2.11.2 Important Queue Methods

Method / Syntax	Purpose
`insert(index, value)`	Inserts `value` at the specified index
`delete(index)`	Removes the element at the specified index
`size()`	Returns the current number of elements
`queue[index]`	Accesses an element at a specific index

2.11.3 Indexing

Queue indexes start from `0`.

For example:

```
int queue[$] = {10, 20, 30, 40};
queue[0] // 10
queue[2] // 30
queue.insert(2, 25);
```

Before:

```
{10, 20, 30, 40}
```

After:

```
{10, 20, 25, 30, 40}
```

The element at index `2` and subsequent elements are shifted toward the end.

2.11.4 Delete

`queue.delete(3);` The element at index `3` is removed and subsequent elements shift toward the beginning.

2.11.5 Size

`queue.size()` returns the current number of elements in the queue.

2.11.6 Verification Relevance

Queue methods are useful for:

- Managing expected transactions.
- Inserting transactions at specific positions.

- Removing processed transactions.
- Accessing individual transactions by index.
- Tracking the current number of pending transactions.

2.11.7 Complete Runnable Code

```
module sv_017_queue_methods;

int queue[$];
int value;

initial begin
    // Initialize queue
    queue = '{10, 20, 30, 40};
    $display("Initial queue:");
    $display("queue = %p", queue);

    // Indexing
    $display("\nIndexing:");
    $display("queue[0] = %0d", queue[0]);
    $display("queue[2] = %0d", queue[2]);

    // Insert 25 at index 2
    queue.insert(2, 25);
    $display("\nAfter insert(2, 25):");
    $display("queue = %p", queue);

    // Delete element at index 3
    value = queue[3];
    queue.delete(3);

    $display("\nDeleted value = %0d", value);
    $display("After delete(3):");
    $display("queue = %p", queue);

    // Queue size
    $display("\nQueue size = %0d", queue.size());
end
endmodule
```

2.11.8 Expected Output

```
Initial queue:
queue = '{10, 20, 30, 40}

Indexing:
queue[0] = 10
queue[2] = 30

After insert(2, 25):
```

```
queue = '{10, 20, 25, 30, 40}

Deleted value = 30
After delete(3):
queue = '{10, 20, 25, 40}

Queue size = 4
```

2.11.9 Key Takeaways

- `queue[index]` accesses an individual element.
- `insert(index, value)` adds an element at a specific position.
- `delete(index)` removes an element from a specific position.
- `size()` returns the current queue length.
- These methods make queues flexible for SystemVerilog verification environments.

2.12 SV-018 — Associative Arrays

2.12.1 Introduction

An **associative array** is a SystemVerilog collection that stores elements using an index or key. Unlike fixed and dynamic arrays, its indexes do not need to form a continuous range. Associative arrays are especially useful when the index space is large or **sparse**, because storage is allocated for the entries that are actually used.

2.12.2 Integer Indexing

An associative array can use an integer as its index:

```
int scores[int];
scores[10] = 95; scores[25] = 88; scores[100] = 76;
```

The keys can be ‘10‘, ‘25‘, and ‘100‘ without requiring elements for all values between them.
String Indexing - A string can also be used as the key:

```
marks["Math"]    = 95;
marks["Physics"] = 88;
marks["Verilog"] = 92;
```

This allows data to be associated with meaningful names rather than numeric indexes.

2.12.3 Sparse Storage

Associative arrays are useful for sparse data because only the entries that are actually assigned need to be stored.

For example: `scores[10] = 95 scores[25] = 88 scores[100] = 76` contains only three stored entries even though the highest index is `100`.

The `size()` method returns the number of stored entries: `scores.size()`

2.12.4 Verification Relevance

Associative arrays are useful in verification for:

- Lookup tables.
- Mapping IDs to transactions.
- Storing expected values using transaction IDs.
- Maintaining sparse data sets.
- Associating meaningful string names with values.
- Scoreboards and reference-model data structures.

2.12.5 Complete Runnable Code

```
module sv_018_associative_arrays;
int scores[int];      // Integer-indexed associative array
int marks[string];    // String-indexed associative array

initial begin
    // Integer indexing
```

```
    scores[10] = 95;
    scores[25] = 88;
    scores[100] = 76;
    $display("Integer-indexed array:");
    $display("scores[10]  = %0d", scores[10]);
    $display("scores[25]  = %0d", scores[25]);
    $display("scores[100] = %0d", scores[100]);

    // String indexing
    marks["Math"]    = 95;
    marks["Physics"] = 88;
    marks["Verilog"] = 92;
    $display("\nString-indexed array:");
    $display("marks[Math]    = %0d", marks["Math"]);
    $display("marks[Physics] = %0d", marks["Physics"]);
    $display("marks[Verilog] = %0d", marks["Verilog"]);

    // Sparse storage
    $display("\nSparse storage:");
    $display("scores.size() = %0d", scores.size());
    $display("marks.size()  = %0d", marks.size());
end
endmodule
```

2.12.6 Expected Output

```
Integer-indexed array:
scores[10]  = 95
scores[25]  = 88
scores[100] = 76

String-indexed array:
marks[Math]    = 95
marks[Physics] = 88
marks[Verilog] = 92

Sparse storage:
scores.size() = 3
marks.size()  = 3
```

2.12.7 Key Takeaways

- Associative arrays use **keys/indexes** rather than fixed positions.
- `int array[int]` creates an integer-indexed associative array.
- `int array[string]` creates a string-indexed associative array.
- Keys do not have to be consecutive.
- This makes associative arrays suitable for **sparse storage**.
- `size()` returns the number of currently stored entries.
- Associative arrays are particularly valuable in verification for lookup tables, transaction tracking, and scoreboard/reference-model data.

2.13 SV-019 — Associative Array Methods

2.13.1 Introduction

SystemVerilog provides built-in methods for inspecting, traversing, and modifying associative arrays. These methods are especially useful because associative-array indexes can be sparse and are not necessarily consecutive.

2.13.2 Important Methods

Method	Purpose
`first()`	Returns the first index
`last()`	Returns the last index
`next(index)`	Returns the next index after the specified index
`prev(index)`	Returns the previous index before the specified index
`exists(index)`	Returns `1` if the index exists, otherwise `0`
`num()`	Returns the number of stored elements
`delete(index)`	Deletes the element at the specified index

2.13.3 `first()` and `last()`

For an associative array:

```
data[10] = 100; data[20] = 200; data[50] = 500; data[80] = 800;
data.first()  // returns 10
data.last()   // returns 80
```

These methods provide the boundary indexes without requiring knowledge of the indexes beforehand.

2.13.4 `next()` and `prev()`

`next(index)` finds the next existing index.

For example:

```
lstinline!data.next(20)
returns lstinline!50
```

Similarly:

```
lstinline!data.prev(50)
returns lstinline!20
```

These methods can be used to traverse an associative array in index order.

2.13.5 `exists()`

```
lstinline!exists(index)
```

checks whether a particular index is present:

```
lstinline!data.exists(20)  // 1
data.exists(30)  // 0
```

This is useful before accessing an associative-array element when the index may not exist.

2.13.6 `num()`

```
lstinline!num()
```

returns the number of currently stored elements:

```
lstinline!data.num()!}
```

If four entries are stored, `num()` returns `4`.

2.13.7 `delete()`

An individual entry can be removed using:

```
data.delete(50);
```

After deletion, index `50` no longer exists.

2.13.8 Verification Relevance

Associative-array methods are useful in verification for:

- Traversing sparse transaction databases.
- Checking whether transaction IDs exist.
- Finding the first or last pending transaction.
- Moving between transaction indexes.
- Removing processed transactions.
- Managing scoreboard and reference-model data.

2.13.9 Complete Runnable Code

```
module sv_019_associative_array_methods;

int data[int];
int index;

initial begin
    // Create sparse associative array
    data[10] = 100;
    data[20] = 200;
    data[50] = 500;
    data[80] = 800;
```

```
12
13      $display("Initial number of elements = %0d", data.num());
14
15      // first() and last() update the index variable
16      data.first(index);
17      $display("First index = %0d", index);
18
19      data.last(index);
20      $display("Last index  = %0d", index);
21
22      // Check whether an index exists
23      $display("\nExists:");
24      $display("data[20] exists = %0d", data.exists(20));
25      $display("data[30] exists = %0d", data.exists(30));
26
27      // Traverse using first() and next()
28      $display("\nForward traversal:");
29
30      data.first(index);
31
32      while (1) begin
33          $display("data[%0d] = %0d", index, data[index]);
34
35          if (!data.next(index))
36              break;
37      end
38
39      // Previous index
40      index = 50;
41
42      if (data.prev(index))
43          $display("\nPrevious index before 50 = %0d", index);
44
45      // Delete one entry
46      data.delete(50);
47
48      $display("\nAfter delete(50):");
49      $display("Number of elements = %0d", data.num());
50      $display("Exists(50) = %0d", data.exists(50));
51  end
52
53  endmodule
```

2.13.10 Expected Output

```
1  Initial number of elements = 4
2  First index = 10
3  Last index  = 80
4
5  Exists:
6  data[20] exists = 1
```

```
data[30] exists = 0

Forward traversal:
data[10] = 100
data[20] = 200
data[50] = 500
data[80] = 800

Previous index before 50 = 20

After delete(50):
Number of elements = 3
Exists(50) = 0
```

2.13.11 Key Takeaways

- `first()` and `last()` find the boundary indexes.
- `next()` and `prev()` allow traversal through existing indexes.
- `exists()` checks whether a specific index is present.
- `num()` returns the number of stored entries.
- `delete(index)` removes a specific associative-array entry.
- `first()` + `next()` is a useful pattern for forward traversal of sparse associative arrays.
- These methods are important for verification data structures such as scoreboards, transaction tracking, and reference models.

2.14 SV-020 — Array Locator Methods

2.14.1 Introduction

SystemVerilog array locator methods search an array using a specified condition. They provide a concise alternative to manually iterating through an array with a loop.

The main methods covered here are the following.

Method	Purpose
`find()`	Returns all matching elements
`find_index()`	Returns indexes of all matching elements
`find_first()`	Returns the first matching element
`find_last()`	Returns the last matching element

The search condition is specified using a `with` clause.

2.14.2 `find()`

`find()` returns a queue containing **all elements** that satisfy the condition.

```
found = data.find(x) with (x >= 30);
```

For:

```
{10, 25, 30, 45, 50, 65}
```

the result is:

```
{30, 45, 50, 65}
```

2.14.3 `find_index()`

`find_index()` returns a queue containing the **indexes** of all matching elements.

```
indexes = data.find_index(x) with (x >= 30);
```

The matching indexes are:

```
{2, 3, 4, 5}
```

2.14.4 `find_first()`

`find_first()` returns the **first element** that satisfies the condition.

```
first_value = data.find_first(x) with (x >= 30);
```

Result:

```
30
```

2.14.5 `find_last()`

`find_last()` returns the **last element** that satisfies the condition.

```
last_value = data.find_last(x) with (x >= 30);
```

Result:

```
65
```

2.14.6 The `with` Clause

The expression inside `with` defines the search condition:

```
data.find(x) with (x >= 30);
```

Here, `x` represents each array element being examined.

Other conditions can also be used:

```
data.find(x) with (x == 50);
data.find(x) with (x < 30);
data.find(x) with ((x % 2) == 0);
```

2.14.7 Verification Relevance

Array locator methods are useful in verification for:

- Searching transaction collections.
- Finding expected values.
- Locating matching data.
- Checking whether specific conditions are satisfied.
- Quickly identifying the first or last matching transaction.
- Reducing manual search loops in testbench code.

2.14.8 Complete Runnable Code

```
module sv_020_array_locator_methods;
int data[6] = '{10, 25, 30, 45, 50, 65};

int found[$];
int indexes[$];
int first_value[$];
int last_value[$];

initial begin
    $display("Array:");
    foreach (data[i])
        $display("data[%0d] = %0d", i, data[i]);

    // Find all matching elements
    found = data.find(x) with (x >= 30);

    $display("\nfind():");
    $display("Matching values = %p", found);

    // Find indexes of all matching elements
    indexes = data.find_index(x) with (x >= 30);

    $display("\nfind_index():");
    $display("Matching indexes = %p", indexes);

    // Find first matching element
```

```
    first_value = data.find_first(x) with (x >= 30);

    $display("\nfind_first():");
    $display("First matching value = %p", first_value);

    // Find last matching element
    last_value = data.find_last(x) with (x >= 30);

    $display("\nfind_last():");
    $display("Last matching value = %p", last_value);
end

endmodule
```

2.14.9 Expected Output

```
Array:
data[0] = 10
data[1] = 25
data[2] = 30
data[3] = 45
data[4] = 50
data[5] = 65

find():
Matching values = '{30, 45, 50, 65}

find_index():
Matching indexes = '{2, 3, 4, 5}

find_first():
First matching value = '{30}

find_last():
Last matching value = '{65}
```

2.14.10 Key Takeaways

- `find()` returns a queue of **matching values**.
- `find_index()` returns a queue of **matching indexes**.
- `find_first()` returns the **first matching value**.
- `find_last()` returns the **last matching value**.
- The `with` clause defines the search condition.
- Array locator methods make verification code more concise when searching transactions, expected data, or other collections.

2.15 SV-021 — Array Ordering

2.15.1 Introduction

SystemVerilog provides built-in methods for rearranging elements of an unpacked array.

The four important array ordering methods are:

Method	Purpose
`sort()`	Sorts elements in ascending order
`rsort()`	Sorts elements in descending order
`reverse()`	Reverses the current element order
`shuffle()`	Randomly rearranges the elements

These methods are useful in verification when test data or transactions need to be sorted, reversed, or randomly reordered.

2.15.2 `sort()`

`sort()` arranges the elements in ascending order.

```
data.sort();
```

Example: `{40, 10, 60, 20, 50, 30}`

becomes: `{10, 20, 30, 40, 50, 60}`

2.15.3 `rsort()`

`rsort()` arranges the elements in descending order.

```
data.rsort();
```

Result: `{60, 50, 40, 30, 20, 10}`

2.15.4 `reverse()`

`reverse()` reverses the current ordering of the elements.

```
data.reverse();
```

For example: `{60, 50, 40, 30, 20, 10}`

becomes: `{10, 20, 30, 40, 50, 60}`

It does **not** sort the array; it simply reverses its current order.

2.15.5 `shuffle()`

`shuffle()` randomly rearranges the elements:

```
data.shuffle();
```

The exact result can differ between simulation runs.

For example: `{30, 60, 10, 50, 20, 40}`

The elements remain the same, but their order changes.

2.15.6 Verification Relevance

Array ordering methods are useful for:

- Sorting expected transaction data.
- Organizing test data.
- Reversing transaction sequences.
- Randomizing the order of stimulus.
- Preparing arrays for comparison.
- Creating varied transaction sequences during verification.

2.15.7 Complete Runnable Code

```
module sv_021_array_ordering;

int data[6] = '{40, 10, 60, 20, 50, 30};

initial begin
    $display("Original array:");
    $display("data = %p", data);

    // Sort in ascending order
    data.sort();

    $display("\nAfter sort():");
    $display("data = %p", data);

    // Sort in descending order
    data.rsort();

    $display("\nAfter rsort():");
    $display("data = %p", data);

    // Reverse current order
    data.reverse();

    $display("\nAfter reverse():");
    $display("data = %p", data);

    // Randomly shuffle elements
    data.shuffle();

    $display("\nAfter shuffle():");
    $display("data = %p", data);
end

endmodule
```

2.15.8 Expected Output

```
Original array:
data = '{40, 10, 60, 20, 50, 30}

After sort():
data = '{10, 20, 30, 40, 50, 60}

After rsort():
data = '{60, 50, 40, 30, 20, 10}

After reverse():
data = '{10, 20, 30, 40, 50, 60}

After shuffle():
data = '{...random order...}
```

The `shuffle()` result is intentionally not fixed because it randomly rearranges the elements on each simulation run.

2.15.9 Key Takeaways

- `sort()` → ascending order.
- `rsort()` → descending order.
- `reverse()` → reverses the current order.
- `shuffle()` → randomly rearranges elements.
- These methods modify the array directly.
- `shuffle()` can produce a different ordering on different simulation runs.
- Array ordering methods are useful for efficient stimulus and transaction manipulation in verification.

2.16 SV-022 — Array Reduction

2.16.1 Introduction

SystemVerilog array reduction methods combine all elements of an array into a single result.

The five important reduction methods are:

Method	Operation
`sum()`	Adds all elements
`product()`	Multiplies all elements
`and()`	Bitwise AND of all elements
`or()`	Bitwise OR of all elements
`xor()`	Bitwise XOR of all elements

2.16.2 `sum()`

`sum()` adds all elements:

```
sum_result = data.sum();
```

For:

```
{12, 10, 6}
```

the result is:

```
12 + 10 + 6 = 28
```

2.16.3 `product()`

`product()` multiplies all elements:

```
product_result = data.product();
```

For the same array:

```
12 * 10 * 6 = 720
```

2.16.4 `and()`

`and()` performs a bitwise AND across all elements:

```
and_result = data.and();
```

For the example:

```
1100
1010
0110
----
0000
```

Therefore:

```
and = 0000
```

2.16.5 or()

or() performs a bitwise OR across all elements:

```
or_result = data.or();
```

```
1100
1010
0110
----
1110
```

Therefore:

```
or = 1110
```

2.16.6 xor()

xor() performs a bitwise XOR across all elements:

```
xor_result = data.xor();
```

```
1100
1010
0110
----
0000
```

Therefore:

```
xor = 0000
```

2.16.7 Verification Relevance

Array reduction methods are useful in verification for:

- Calculating totals of transaction fields.
- Generating expected arithmetic results.
- Checking combined bit conditions.
- Calculating parity-like values with XOR.
- Creating compact reference-model calculations.
- Reducing repetitive loops in testbench code.

2.16.8 Complete Runnable Code

```
module sv_022_array_reduction;

logic [3:0] data[3] = '{4'b1100, 4'b1010, 4'b0110};

int sum_result;
int product_result;
logic [3:0] and_result;
logic [3:0] or_result;
logic [3:0] xor_result;
```

```
10
11 initial begin
12     $display("Array:");
13     foreach (data[i])
14         $display("data[%0d] = %b", i, data[i]);
15
16     // Arithmetic reductions
17     sum_result     = data.sum();
18     product_result = data.product();
19
20     // Bitwise reductions
21     and_result = data.and();
22     or_result  = data.or();
23     xor_result = data.xor();
24
25     $display("\nReduction Results:");
26     $display("sum     = %0d", sum_result);
27     $display("product = %0d", product_result);
28     $display("and     = %b", and_result);
29     $display("or      = %b", or_result);
30     $display("xor     = %b", xor_result);
31 end
32
33 endmodule
```

2.16.9 Expected Output

```
1  Array:
2  data[0] = 1100
3  data[1] = 1010
4  data[2] = 0110
5
6  Reduction Results:
7  sum     = 28
8  product = 720
9  and     = 0000
10 or      = 1110
11 xor     = 0000
```

2.16.10 Key Takeaways

- `sum()` → adds all array elements.
- `product()` → multiplies all array elements.
- `and()` → bitwise AND across all elements.
- `or()` → bitwise OR across all elements.
- `xor()` → bitwise XOR across all elements.
- Reduction methods provide concise alternatives to manually iterating through arrays.
- They are useful for arithmetic calculations and bit-level checks in verification.

2.17 SV-023 — Array Iteration

2.17.1 Introduction

SystemVerilog provides the `foreach` loop for traversing array elements conveniently.

Instead of manually writing an index-based loop such as:

`for (int i = 0; i < 5; i++)`

`foreach` automatically iterates over the valid indexes of the array.

2.17.2 `foreach`

Basic syntax:

`foreach (array[index]) begin // statements end`

For example: `foreach (data[i]) $display("data[%0d] = %0d", i, data[i]);`

Here, `i` is the **iterator variable**. During each iteration, it represents the current index.

For: `data = {10, 20, 30, 40, 50}`

the iterator takes the values: `i = 0 i = 1 i = 2 i = 3 i = 4`

2.17.3 Iterator Variables

The iterator variable is declared by the `foreach` statement: `foreach (data[i])`

It can then be used to access both the current index and element:

`data[i]`

For multidimensional arrays, multiple iterator variables can be used:

`foreach (matrix[i, j]) $display("%0d", matrix[i][j]);`

2.17.4 Array Traversal

`foreach` automatically visits every valid element of the array. This makes it useful for:

- Displaying array contents.
- Calculating values.
- Checking elements.
- Modifying elements.
- Processing transaction arrays in verification.

For example, calculating a sum:

```
1 sum = 0;
2 foreach (data[i]) sum += data[i];
```

2.17.5 Verification Relevance

Array iteration is frequently used in verification for:

- Traversing transaction arrays.
- Checking expected values.
- Processing stimulus.
- Calculating reference-model results.
- Inspecting queues and other SystemVerilog collections.
- Writing concise testbench code without manually managing indexes.

2.17.6 Complete Runnable Code

```
module sv_023_array_iteration;

    int data[5] = '{10, 20, 30, 40, 50};
    int sum;

    initial begin
        $display("Array traversal:");

        // Iterator variable i represents the array index
        foreach (data[i]) begin
            $display("data[%0d] = %0d", i, data[i]);
        end

        // Use iterator to calculate the sum
        sum = 0;

        foreach (data[i]) begin
            sum += data[i];
        end

        $display("\nSum = %0d", sum);
    end

endmodule
```

2.17.7 Expected Output

```
Array traversal: data[0] = 10 data[1] = 20 data[2] = 30 data[3] = 40 data[4] = 50

Sum = 150
```

2.17.8 Key Takeaways

- ‘foreach‘ provides convenient array traversal.
- The variable inside ‘foreach‘ acts as the **iterator variable**.
- The iterator represents the current array index.
- ‘data[i]‘ accesses the current element.
- Multiple iterator variables can be used for multidimensional arrays.
- ‘foreach‘ is widely used in verification because it produces concise and readable array-processing code.

2.18 SV-024—Assignment-Patterns

2.18.1 Introduction

Assignment patterns use ‘’ ‘ to assign values to aggregate data types such as arrays and structures. They provide a compact way to initialize multiple elements at once.

Simple Assignment Pattern:

```
int data[4];
data = '{10, 20, 30, 40};
```

The values are assigned sequentially to the array elements.

2.18.2 Nested Assignment Patterns

For multidimensional arrays, assignment patterns can be nested:

```
int matrix[2][3];
matrix = '{'{1, 2, 3}, '{4, 5, 6}};
```

The outer pattern represents the first dimension, while each inner pattern represents one row.

2.18.3 Struct Assignment Pattern

Assignment patterns can also assign values to structure members:

```
typedef struct {
    string name;
    int age;
} person_t;
person_t person;
person = '{"Bibhu", 21};
```

The values are assigned according to the structure member order.

2.18.4 Verification Relevance

Assignment patterns are useful in verification for quickly creating:

- Test data arrays
- Multidimensional test vectors
- Transaction fields
- Structure-based configuration data
- Initial values for stimulus

2.18.5 Complete Runnable Code

```
module sv_024_assignment_patterns;

    typedef struct {
        string name;
        int age;
```

```
    } person_t;

    int data[4];
    int matrix[2][3];
    person_t person;

    initial begin

        // Simple array assignment pattern
        data = '{10, 20, 30, 40};

        $display("Simple Assignment Pattern:");
        $display("data = %p", data);

        // Nested assignment pattern for 2D array
        matrix = '{'{1, 2, 3}, '{4, 5, 6}};

        $display("\nNested Assignment Pattern:");
        $display("matrix = %p", matrix);

        // Assignment pattern for a structure
        person = '{"Bibhu", 21};

        $display("\nStruct Assignment Pattern:");
        $display("name = %s", person.name);
        $display("age  = %0d", person.age);
    end
endmodule
```

2.18.6 Expected Output

```
Simple Assignment Pattern:
data = '{10,20,30,40}

Nested Assignment Pattern:
matrix = '{'{1,2,3},'{4,5,6}}

Struct Assignment Pattern:
name = Bibhu
age  = 21
```